

Monitoring & Observability in AWS

• CloudWatch • CloudTrail • X-Ray • EventBridge

Minh Huỳnh
Solution Architect - TechX



SESSION 01

**Monitoring & Observability
Overview**

SESSION 02

**CloudWatch Metrics &
Dashboards**

SESSION 03

**CloudWatch Alarms &
Auto-Alerting**

SESSION 04

**CloudWatch Logs
Management**

SESSION 05

**CloudTrail — Security &
Audit**

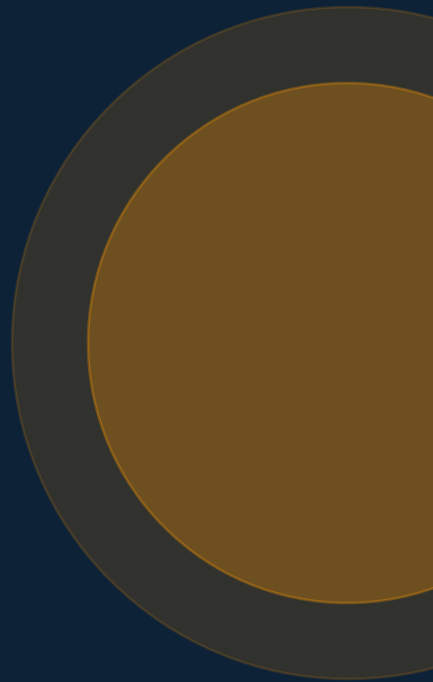
SESSION 06

X-Ray — Distributed Tracing

SESSION 01

Monitoring & Observability

Understanding the foundational concepts of cloud system monitoring



Monitoring vs. Observability — What's the Difference?

SESSION 01



MONITORING

- ▶ Is the system running?
- ▶ Are response times normal?
- ▶ Is CPU below threshold?

FOCUS

Tells you WHEN something is wrong

TOOLS

Dashboards, Thresholds, Alerts



OBSERVABILITY

- ▶ WHY is it running slow?
- ▶ Where did the request fail?
- ▶ What changed at 2 AM?

FOCUS

Tells you WHY something went wrong

TOOLS

Logs, Traces, Metrics (3 Pillars)



Metrics

Numbers over time

- CPU utilization, memory, request rate
- Aggregated numerical data
- Ideal for dashboards & alerts
- Examples: CloudWatch Metrics, Prometheus



Logs

Event records

- Timestamped event messages
- Raw, unstructured or structured text
- Ideal for root-cause investigation
- Examples: CloudWatch Logs, ELK Stack



Traces

Request journeys

- End-to-end request lifecycle
- Spans across microservices
- Ideal for distributed systems
- Examples: AWS X-Ray, Jaeger

SLA — Service Level Agreement

A formal contract between a service provider and customer defining guaranteed uptime and penalties for breach.
Example: "AWS guarantees 99.99% monthly uptime for EC2. If we fall below, we credit your account."

SLO — Service Level Objective

An internal target your team sets for service quality — more specific and engineering-facing than an SLA.
Example: "Our checkout API must respond in < 200ms for 99.5% of requests over any 30-day window."

 *Key Insight: SLAs are external & legal — SLOs are internal & engineering. You cannot meet your SLA without first defining and tracking your SLOs.*

Service Level Objectives (SLO) [Info](#)

Monitoring account

View data for:

All accounts ▼

1h

12h

3h 📅

UTC timezone ▼

☐ Show selected SLO from beginning of current inter

▼ SLO selected: Latency for Retail Service Dev Enironment

SLI: retail_service_dev/L...

SLO attainment

Error budget



■ Good requests ■ Bad requests

%



■ SLO attainment

%



■ Error budget

Service Level Objectives (SLO) (19) [Info](#)[Actions](#) ▼[Create SLO](#)

< 1 >





CloudWatch

Metrics, Logs, Alarms, Dashboards



CloudTrail

API call audit logs & compliance



EventBridge

Event routing & automation



X-Ray

Distributed tracing for microservices



GuardDuty

Threat detection & security



Config

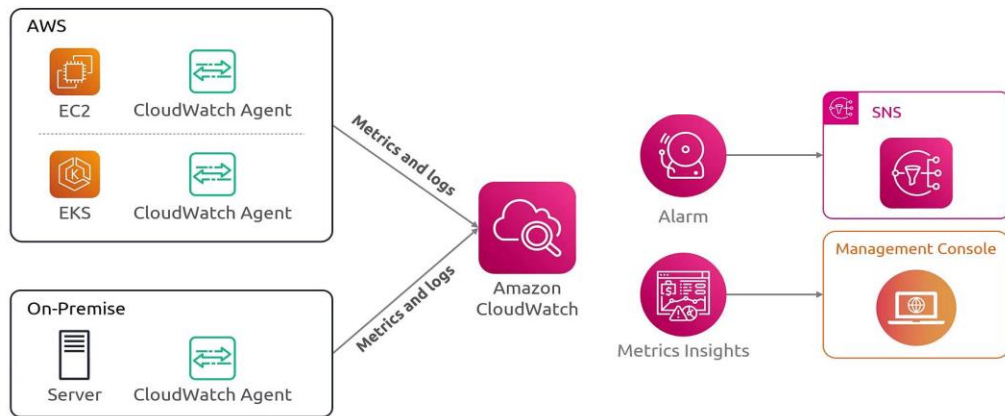
Resource configuration compliance

SESSION 02

CloudWatch Metrics & Dashboards

Collecting, visualizing, and understanding AWS service metrics

- CloudWatch provides metrics for every services in AWS
- Metric is a variable to monitor (CPUUtilization, NetworkIn...)
- Metrics belong to namespaces
- Dimension is an attribute of a metric (instance id, environment, etc...).
- Up to 30 dimensions per metric
- Metrics have timestamps
- Can create CloudWatch dashboards of metrics
- Can create CloudWatch Custom Metrics (for the RAM for example)





Host-Level Metrics (Default)

Automatically collected by AWS — no setup needed

✓ CPU Utilization

✓ Network In / Out

✓ Disk Read / Write I/O

✓ Status Check (System & Instance)

✓ EBS Throughput



FREE — included with all EC2 instances



Custom Metrics (CloudWatch Agent)

Requires CloudWatch Agent installed on EC2

✓ Memory Utilization (%)

✓ Disk Space Used / Available

✓ Swap Usage

✓ Process Count

✓ Custom App Metrics (via API)



Additional cost per custom metric published

1

Install the Agent Package

```
sudo yum install amazon-cloudwatch-agent -y
# Or for Ubuntu:
sudo apt-get install amazon-cloudwatch-agent
```

2

Run Configuration Wizard

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/
amazon-cloudwatch-agent-config-wizard
```

 Prerequisite: EC2 IAM Role must have CloudWatchAgentServerPolicy attached

3

Start the Agent

```
sudo systemctl enable amazon-cloudwatch-agent
sudo systemctl start amazon-cloudwatch-agent
```

4

Verify & Check Status

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/
amazon-cloudwatch-agent-ctl -m ec2 -a status
```

Building a CloudWatch Dashboard — EC2 Monitoring

SESSION 02

CPU Utilization (%)



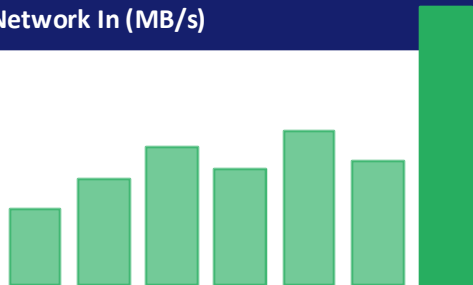
▲ 67%

Memory Usage (%)



▲ 82%

Network In (MB/s)



→ 145

Network Out (MB/s)

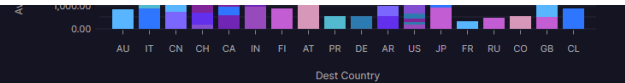
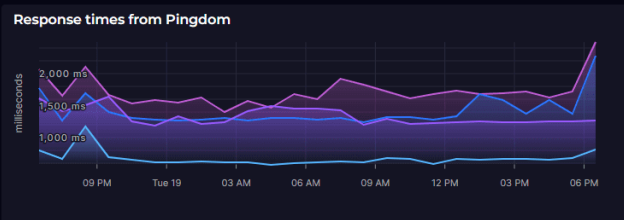
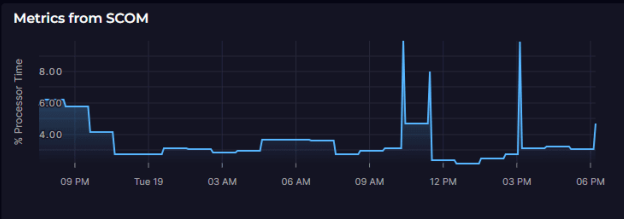
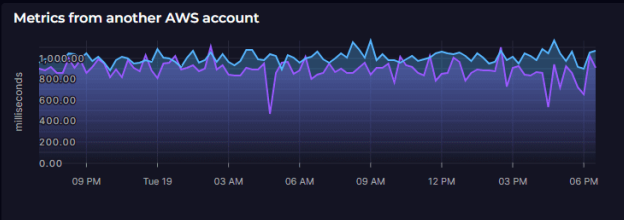
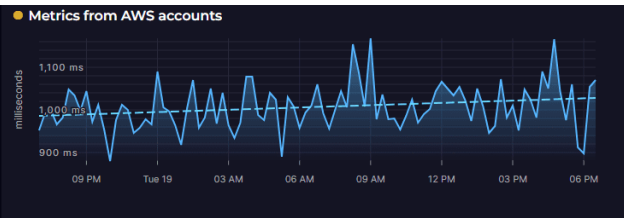


↓ 98

Disk Read I/O



▲ 230



Pipeline runs from CircleCI and Azure DevOps

12013 shipping-service	12012 shipping-service	12000 shipping-service
12013 inventory-service	12008 inventory-service	23485 account-service
23483 account-service	23474 account-service	23470 account-service
23468 account-service	6592 order-service	6590 order-service
6581 order-service	6576 order-service	

PRs and incidents from GitHub

ID	Num...	Title	State	User	Created At	Updated At
1779505450	3,830	Fixing failed linting checks	open	autosquaredup	3/19/2024, 3:43:00 PM	3/19/2024, 3:43:00 PM
1779505455	1,727	Fixing failed linting checks	open	autosquaredup	3/19/2024, 3:43:00 PM	3/19/2024, 3:43:00 PM
1779505458	9,548	Extended error handling to cover API methods	open	autosquaredup	3/19/2024, 3:43:00 PM	3/19/2024, 3:43:00 PM
1778557613	3,817	Tagging release for deployment	open	autosquaredup	3/19/2024, 3:43:20 AM	3/19/2024, 3:43:00 PM

Per page 10

Showing 1 - 10 of 38



SESSION 03

CloudWatch Alarms & Auto-Alerting

Building intelligent alerting systems that respond automatically

CloudWatch Alarm States — OK, ALARM, INSUFFICIENT_DATA

SESSION 03

OK



Metric is within the defined threshold.
System is healthy.
Example: CPU at 45% — threshold is 80%. All good!

ALARM



Metric has breached the threshold.
Action triggered.
Example: CPU at 92% for 5 minutes — alert sent via SNS!

INSUFFICIENT DATA



Not enough data to evaluate. Usually occurs on startup.
Example: Agent just started — metric data not yet available.



Evaluation Period: CloudWatch checks the metric every period (e.g., 1 min). You define how many consecutive periods must breach before triggering ALARM (e.g., 3 out of 5).

Alarms (27)

☐ Hide Auto Scaling alarms

Clear selection



Create composite alarm

Actions ▼

Create alarm

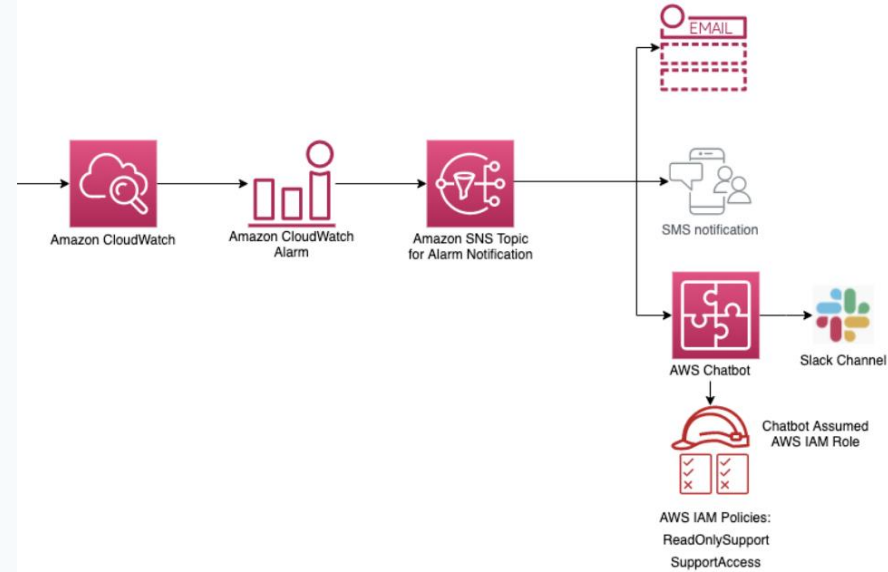
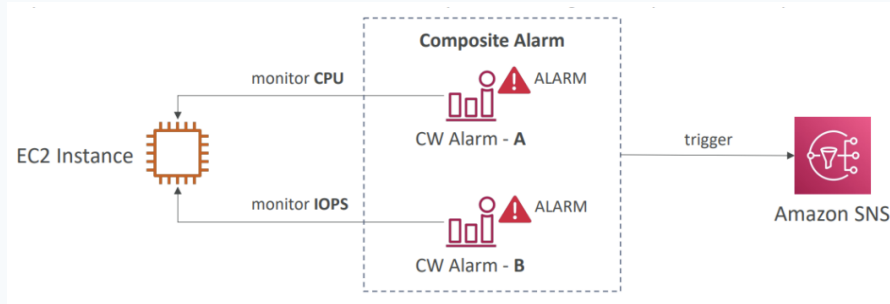
Any state ▼

Any type ▼

< 1 >



☐	Name ▼	State ▼	Last state update ▼	Conditions	Actions ▼
☐	awsec2-i-0fc458fad8fc7fac2-LessThanOrEqualToThreshold-CPUCreditBalance	✔ OK	2021-07-04 10:06:27	CPUCreditBalance <= 1 for 1 datapoints within 5 minutes	No actions
☐	TargetTracking-table/TextNote-AlarmLow-d4b5d3ff-9b62-4189-9f9c-b6fafd379abe	⚠ In alarm	2021-06-28 16:23:46	ConsumedWriteCapacityUnits < 30 for 15 datapoints within 15 minutes	✔ 1 action(s) enabled
☐	TargetTracking-table/TextNote/index/textNoteSecondaryIndex-AlarmLow-8b591a75-227c-4cb2-be8f-ff3209a4b54e	⚠ In alarm	2021-06-28 16:23:40	ConsumedReadCapacityUnits < 30 for 15 datapoints within 15 minutes	✔ 1 action(s) enabled
☐	TargetTracking-table/TextNote-AlarmLow-a0552914-7d04-4a3c-8dd8-bd0d74cb9d9a	⚠ In alarm	2021-06-28 16:23:21	ConsumedReadCapacityUnits < 30 for 15 datapoints within 15 minutes	✔ 1 action(s) enabled



SNS uses a Publish/Subscribe model: CloudWatch publishes a message to a Topic, and all subscribers receive it simultaneously.



Static Threshold

ALARM if CPU > 80% for 5 minutes

- ✓ Simple to configure
- ✓ Easy to understand
- ✓ Predictable behavior
- ✗ Needs manual tuning
- ✗ High false-positive rate
- ✗ Doesn't adapt to patterns



Anomaly Detection

ALARM if traffic deviates from ML baseline

- ✓ Learns seasonal patterns
- ✓ Adapts to growth trends
- ✓ Reduces false positives
- ✓ Handles peak hours
- ✗ Needs 2 weeks training data
- ✗ Higher cost

Scenario: Send an email alert when EC2 CPU > 80% for 5 consecutive minutes

1

Create SNS Topic & Subscription

Navigate: SNS → Create Topic (Standard) → Add Email Subscription
Confirm subscription via email link

2

Create CloudWatch Alarm

Navigate: CloudWatch → Alarms → Create Alarm
Select Metric: EC2 → Per-Instance → CPUUtilization

3

Configure Alarm Conditions

Condition: Greater than 80%
Period: 5 minutes | Evaluation: 1 out of 1 datapoints

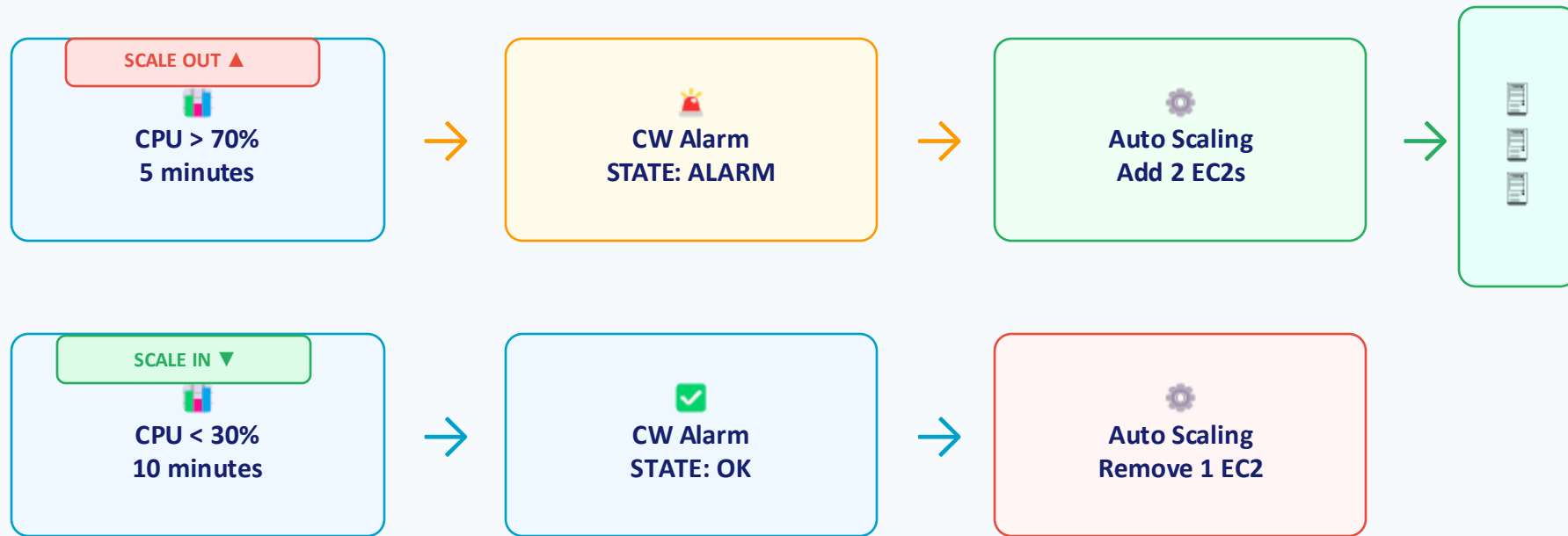
4

Set SNS Notification Action

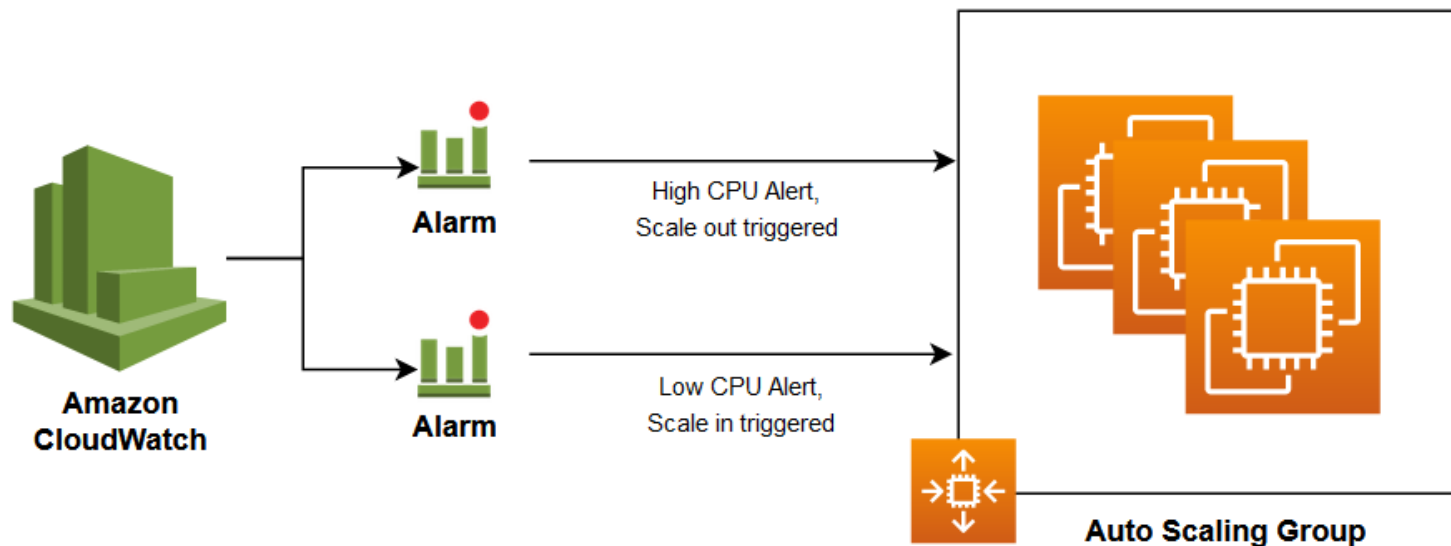
In Alarm state → Add notification → Select your SNS Topic
Optional: Add OK state notification (recovery alert)

Auto Scaling Triggered by CloudWatch Alarms

SESSION 03



🎯 Auto Scaling ensures your app handles load spikes automatically — no manual intervention needed, and you only pay for capacity you actually use.

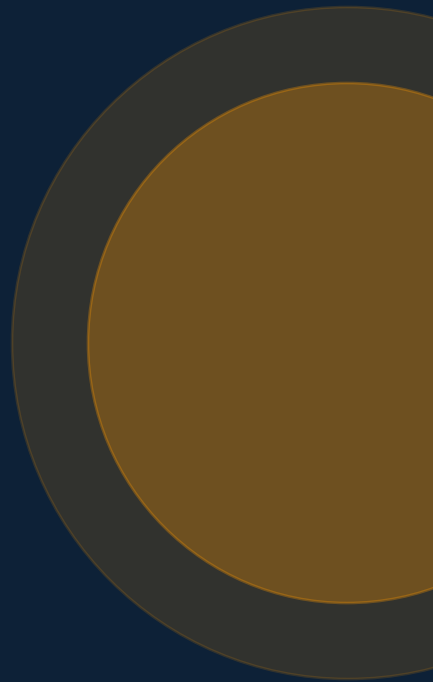


Amazon EC2 Auto Scaling

SESSION 04

CloudWatch Logs Management

Centralized logging: collect, store, search, and analyze system logs



LOG GROUP (e.g., /aws/ec2/nginx-prod)

 Log Stream: i-0abc1234 (EC2 Instance A)

 Log Stream: i-0def5678 (EC2 Instance B)

 Log Stream: i-0ghi9012 (EC2 Instance C)

Retention Policy Options

Period	Use Case
1 day	Debug / dev environments
7 days	Short-term operational logs
30 days	Standard production apps
90 days	Compliance-lite workloads
1 year+	Regulatory / audit logs
Never expire	Critical audit trails



Cost Tip

CloudWatch Logs charges for:

- Data ingested (per GB)
- Data archived (per GB/month)
- Data scanned with Insights queries

Set retention policies to avoid paying for stale logs!

Pushing Application Logs to CloudWatch

SESSION 04



Nginx / Apache

```
# CloudWatch Agent config snippet

log_file = "/var/log/nginx/access.log"
log_group_name = "/aws/ec2/nginx"
```



App Logs (Python/Java)

```
# CloudWatch Agent config snippet

log_file = "/var/log/app/app.log"
log_group_name = "/aws/ec2/application"
```



Docker / ECS Container

```
# CloudWatch Agent config snippet

"logDriver": "awslogs",
"options": {"awslogs-group": "/ecs/app"}
```

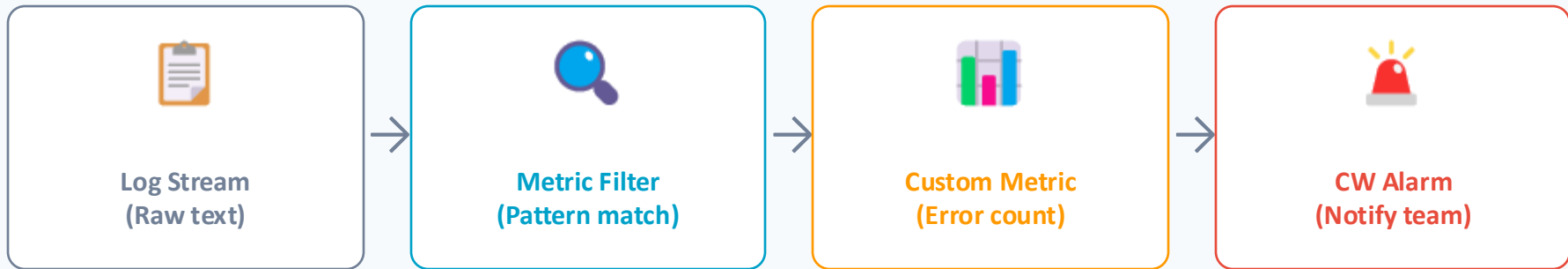


All logs stream to CloudWatch in near-real time and are searchable with CloudWatch Logs Insights (SQL-like query language)

Metric Filters — Turning Log Text into Metrics

SESSION 04

Use Case: Count HTTP 500 errors per minute from Nginx access logs



Sample Log Lines:

```
192.168.1.1 - - [01/Jun/2025] "GET /api/users" 200 512
10.0.0.5 - - [01/Jun/2025] "POST /api/checkout" 500 128
172.16.0.2 - - [01/Jun/2025] "GET /health" 200 48
10.0.0.5 - - [01/Jun/2025] "POST /api/checkout" 500 128
```

Filter Pattern: [ip, dash, user, date, request, statusCode=500, bytes] → Metric:
HTTP_500_Errors, Value: 1

Logs Insights uses a purpose-built query language to analyze and search log data across multiple Log Groups.

Top 10 slowest API requests

```
filter @type = "REPORT"
| sort @duration desc
| limit 10
| display @requestId, @duration, @billedDuration
```

Count errors by status code

```
filter status_code >= 400
| stats count(*) as errorCount by status_code
| sort errorCount desc
```

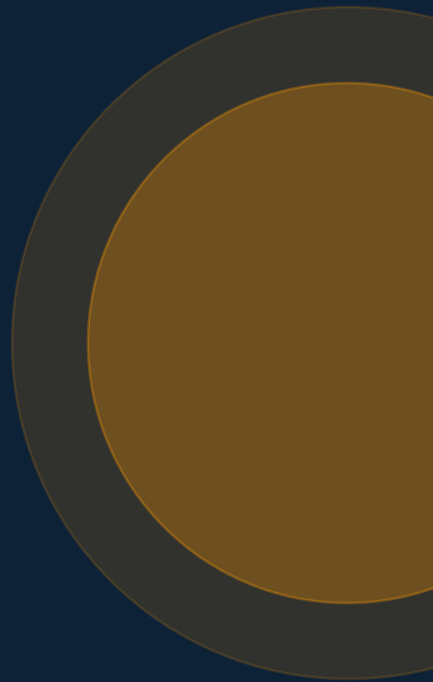
Average Lambda execution time

```
filter @type = "REPORT"
| stats avg(@duration), max(@duration),
  min(@duration) by bin(5m)
```

SESSION 05

CloudTrail — Security & Audit

Track who did what, when, and from where across your AWS account



AWS CloudTrail records every API call made in your AWS account — who made it, what action was performed, when, and from where.



Who?

IAM User, Role, or Service that made the call



What?

AWS API action performed (e.g., CreateInstance, DeleteBucket)



When?

Exact timestamp of the API call (UTC)



Where?

Source IP address and AWS region



Response?

Success or error response code returned by AWS



Management Events

Control-plane operations — actions that manage AWS resources

- ? Who created or terminated an EC2 instance?
- ? Who modified a Security Group rule?
- ? Who created or deleted an IAM user?
- ? Who changed an S3 bucket policy?
- ? Who enabled/disabled CloudTrail itself?

FREE — included in all trails by default

VS



Data Events

Data-plane operations — actions performed ON the data in resources

- ? Who read or downloaded a file from S3?
- ? Who uploaded a new object to S3?
- ? Which Lambda function was invoked?
- ? Who queried a DynamoDB table?
- ? Who deleted an S3 object?

\$ Additional cost — charged per event recorded

CloudWatch vs CloudTrail — Key Differences

SESSION 05

A common exam question — and a real-world skill!

Aspect	CloudWatch 	CloudTrail 
Focus	System performance	User/API activity
Question answered	Is my EC2 CPU too high?	Who deleted my S3 bucket?
Primary use	Monitoring & Alerting	Security & Compliance audit
Data type	Metrics, Logs, Events	API call history (JSON)
Default enabled	Partial (basic metrics)	Yes (90 days, free tier)
Storage	CloudWatch console	S3 bucket (for trail)
Integration	SNS, Auto Scaling, Lambda	EventBridge, CloudWatch Alarms

Security Best Practice: The root account should almost never be used. Alert immediately if it is!

1

Enable CloudTrail & Send Logs to CloudWatch

```
CloudTrail → Trails → Create Trail  
Enable: Log file delivery to CloudWatch Logs group
```

2

Create CloudWatch Metric Filter

```
Filter Pattern: { $.userIdentity.type = "Root" && $.eventType != "AwsServiceEvent" }  
Metric Name: RootAccountLoginCount | Namespace: Security | Value: 1
```

3

Create CloudWatch Alarm

```
Alarm if RootAccountLoginCount >= 1 in any 5-minute period  
This means: ANY single root login should trigger the alarm
```

4

Notify via SNS

```
Add SNS action → notify Security Team immediately via Email + SMS  
Optional: Trigger Lambda to auto-disable root credentials
```

SESSION 06

AWS X-Ray — Distributed Tracing

Debug and optimize microservices by tracing requests end-to-end





Monolith (Easy to debug)

Single App
(All code in one place)

- ✓ One log file
- ✓ One stack trace
- ✓ Easy to follow



Microservices (Hard to debug!)

API Gateway

Order Service

Auth Service

Payment Svc

DynamoDB

Notification

✗ Which service caused the 3-second delay?

Logs are split across 6 services!



Nightmare!

AWS X-Ray traces the complete journey of a request as it travels through your distributed application — across microservices, databases, and queues.

Trace

A complete end-to-end record of one request journey, from the initial entry point to the final response. Contains all Segments.

Segment

One service's contribution to a Trace. Each service (Lambda, EC2, RDS) generates its own Segment with timing data.

Subsegment

A sub-unit within a Segment. Example: a single SQL query or external HTTP call inside a Lambda function.

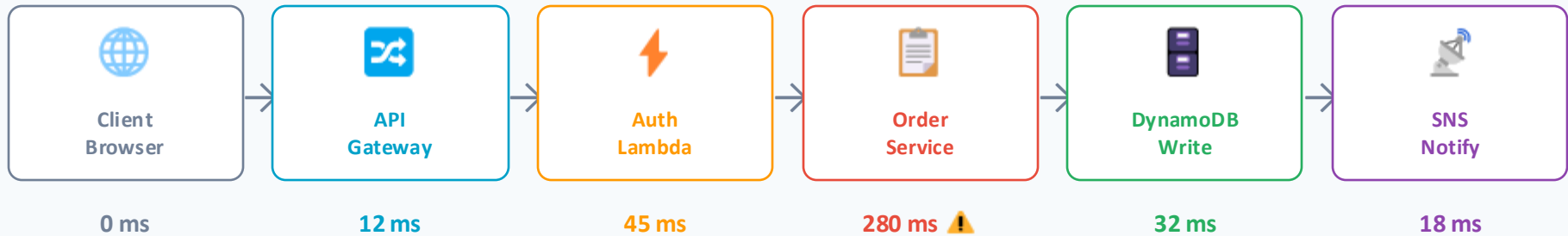
Service Map

X-Ray automatically generates a visual map of all services and their connections — great for architecture discovery.

X-Ray in Action — Request Trace Through Microservices

SESSION 06

User makes a POST /checkout request. X-Ray traces the entire journey:



X-Ray Trace Waterfall (Gantt-style):

Total Request

API Gateway

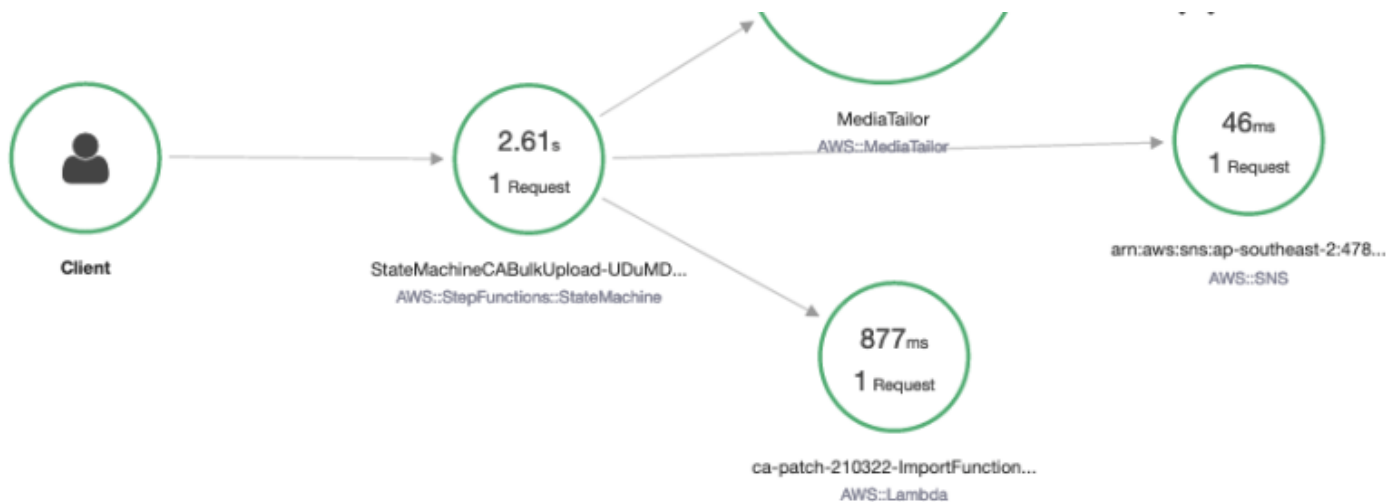
Auth Lambda

Order Service

DynamoDB

SNS Notify





Name	Res.	Duration	Status	0.0ms	200ms	400ms	600ms	800ms	1.0s	1.2s	1.4s	1.6s	1.8
------	------	----------	--------	-------	-------	-------	-------	-------	------	------	------	------	-----

▼ StateMachineCABulkUpload-UDuMDRGVZIDf AWS::StepFunctions::StateMachine

StateMachineCABulkUpload-UDuMDRGVZIDf	-	2.6 sec	✓										
Read CSV from S3 and output JSON	-	929 ms	✓										
Lambda	200	877 ms	✓										
Map	-	1.4 sec	✓										
Iteration 0	-	1.4 sec	✓										
CreateVodSource	-	287 ms	✓										
MediaTailor	200	185 ms	✓										

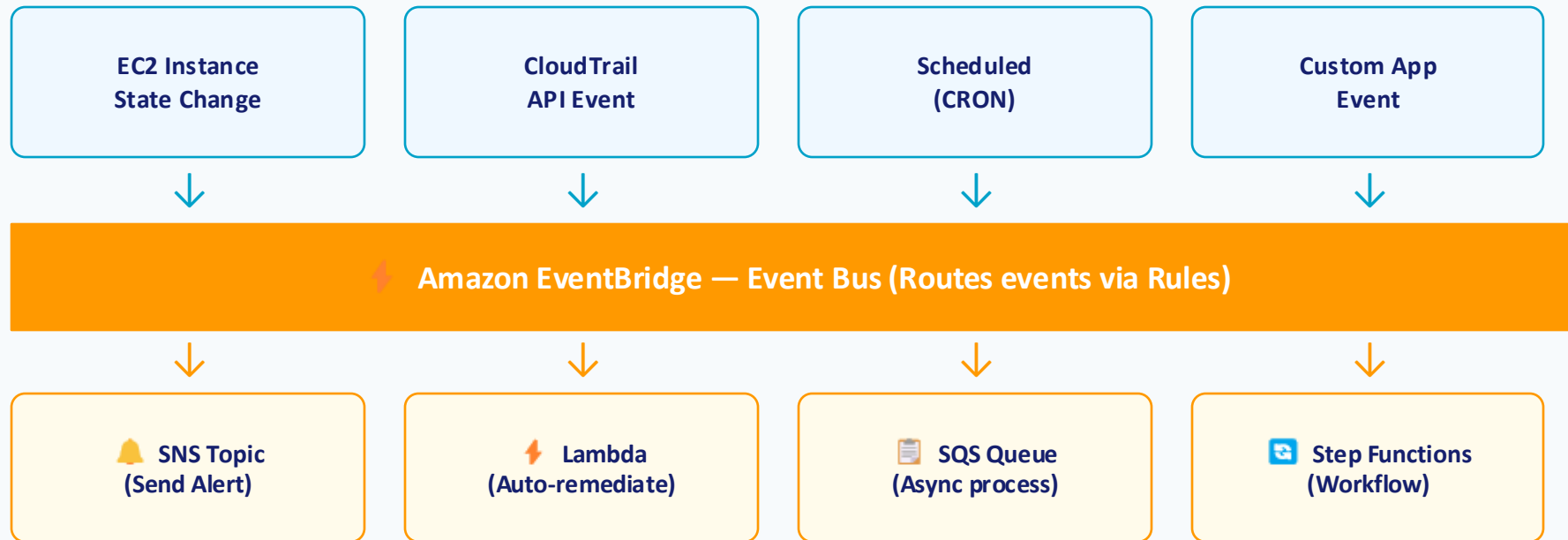
Invoke: ca-patch-210322-ImportFun

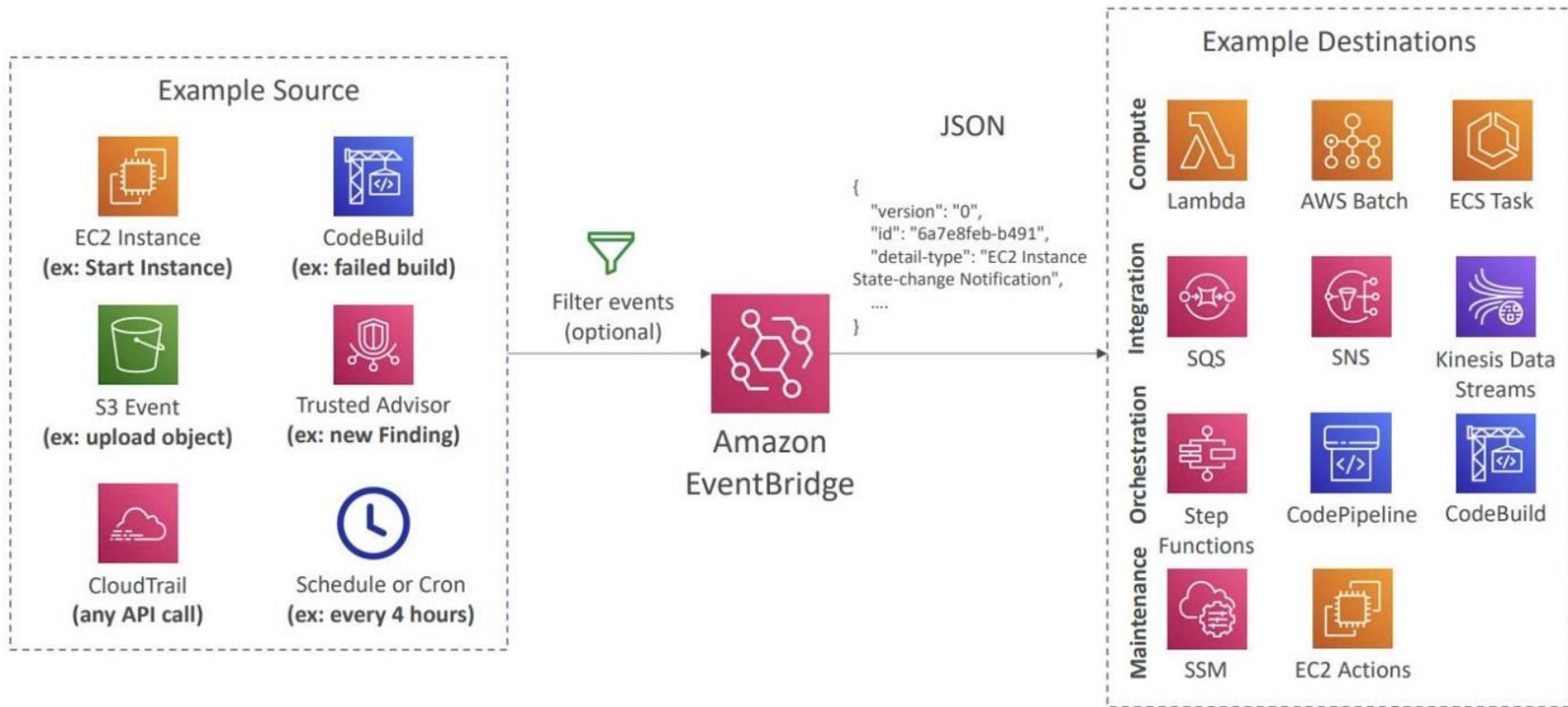
CreateVodSource

Amazon EventBridge — Event-Driven Automation

ADVANCED

EventBridge is a serverless event bus that connects AWS services, SaaS apps, and your custom apps using event-driven rules.





Use Case: AWS Cost Monitoring & Budget Alerts

ADVANCED

Never get a surprise AWS bill again — set up proactive cost alerts.



AWS Budgets

- Set monthly/quarterly cost or usage limits
- Alert at 50%, 80%, 100% of budget
- Forecast alerts (predict before you exceed)
- Action: Auto-stop EC2 or apply SCP
- Integrates with SNS for notifications



Use when you want spending guardrails and automatic actions



Cost Anomaly Detection

- ML-powered — learns your normal spending patterns
- Detects unusual spikes automatically
- No threshold to set — fully adaptive
- Daily or weekly summary emails
- Drill down by service, account, or tag



Use when you want AI to catch unexpected charges you wouldn't think to check

Security Monitoring Best Practices on AWS

ADVANCED

Enable CloudTrail in ALL Regions

CRITICAL

Attackers exploit regions you're not watching. Enable multi-region trail with log file validation.

Enable AWS GuardDuty

HIGH

ML-based threat detection for unusual API calls, cryptocurrency mining, and compromised credentials.

Set S3 Bucket for CloudTrail Logs

HIGH

Store logs in a separate, locked-down S3 bucket with MFA delete enabled and cross-account access.

Alert on IAM Changes

HIGH

Create CloudWatch Alarms for: policy changes, new user creation, access key creation, root login.

Set Log Retention Appropriately

MEDIUM

Security logs: keep at least 1 year. Move to S3 Glacier after 90 days to reduce CloudWatch costs.

Use AWS Security Hub

MEDIUM

Aggregates findings from GuardDuty, Config, Inspector into a single security dashboard.

Course Summary — What You've Learned

SUMMARY

01

Monitoring & Observability

Monitoring vs Observability • 3 Pillars • SLO vs SLA

02

CloudWatch Metrics

Default metrics • Custom metrics • Agent install • Dashboards

03

CloudWatch Alarms

Alarm states • SNS • Static vs Anomaly • Auto Scaling

04

CloudWatch Logs

Log Groups & Streams • Retention • Metric Filters • Insights

05

CloudTrail Security

API audit logs • Mgmt vs Data events • Root login alert

06

X-Ray Tracing

Traces, Segments • Service Map • Bottleneck identification

Thank You!

You're now equipped to build production-grade monitoring systems on AWS

- ✓ Monitor metrics, ship logs, trace requests
- ✓ Automate alerts — don't rely on humans to notice
- ✓ Security monitoring is non-negotiable (CloudTrail on!)
- ✓ Observability = data to answer 'Why is it broken?'